Proj 3: Fixed-Point Arithmetic Operations

Jianhua Liu

Spring 2026

Contents

3.1 Introduction . . . 1
3.1.1 Purpose of the project . . . 1
3.1.2 Learning objectives . . . 1
3.2 Getting started with the project (10 pts) . . . 1
3.3 Programming tasks (PTs) of the project (80 pts) . . . 2
3.3.1 Three function-programming tasks (20 pts each) . . . 2
3.3.2 Test cases . . . 3
3.3.3 The main App (20 pts) . . . 4
3.4 Submission of the work (10 pts) . . . 4

3.1 Introduction

3.1.1 Purpose of the project

The purpose of this project is to reinforce the concepts of fixed-point numbers and their operations introduced in Lecture 13. By implementing these operations in C, students will gain hands-on experience and a deeper understanding of how fixed-point arithmetic works in practice. Through coding and testing, students will solidify their knowledge and improve their ability to apply these concepts in real-world scenarios.

3.1.2 Learning objectives

By the end of this project, you will be able to:

1. Apply the Test-Driven Development (TDD) approach to project development.
2. Develop functions to decode UQm.n and Qm.n data into real numbers.
3. Implement functions to encode real numbers using UQm.n and Qm.n formats.
4. Perform multiplication of real numbers using fixed-point representations

3.2 Getting started with the project (10 pts)

The same as pervious projects, you can use the provided `ca4b_cls_projs` base project. Here, you need to put the source files, which are included in `ee4u_fixed_point_operations-base-files.zip`, in

the `proj3_src` and `proj3_test` folders, respectively. Then, you need to link the C source files to the `proj3_src_test` folder in CubeIDE.

Now, build `Proj3Unity`. This should build without any problem. If you have issues, ask around.

To run the built executables in Renode, you need to create the Renode script files using the examples given in the `rende` folder of the project.

3.3 Programming tasks (PTs) of the project (80 pts)

Here, we practice the programming of Qm.n operation. When m is 0 and n is 15, we will have the commonly used Q15 operation. Note, however, you cannot use the Q15 operation functions provided in the DSP libraries.

3.3.1 Three function-programming tasks (20 pts each)

There are three functions in `ee4u_fixed_point_fns.c`. You need to pass the tests for each function. This can be done using the Unity build on either F412dsc or G431n32.

The base code for these functions are listed below:

```c
#include "ee4u_fixed_point_number_fns.h"

/*
Function for UQm.n and Qm.n decoding. Given the coded data D and the number
    of bits for each field, determine the real number f.

Input:
-   D: encoded data.
-   s: type of representation: 1 for Qm.n (signed); 0 for UQm.n.
-   m: number of bits in the whole number field.
-   n: number of bits in the fraction field.

Return:
-   f: the real number.
*/

__attribute__((weak))
float mp_uq_and_q_mn_decoding(uint32_t D, int s, int m, int n) {
    float f = 0.0;
    return f;
}

/*
Function for UQm.n and Qm.n encoding. Given the real number f and the number
    of bits for each field, determine the real number f.

Input:
-   f: real number to be encoded.
-   s: type of representation: 1 for Qm.n (signed); 0 for UQm.n.
-   m: number of bits in the whole number field.
-   n: number of bits in the fraction field.

Return:
-   D: the encoded data.
*/
```

```

__attribute__((weak))
uint32_t mp_uq_and_q_mn_encoding(float f, int s, int m, int n) {
    uint32_t D = 0;
    return D;
}

/*
Function for Qm.n multiplication. Given real numbers f1 and f1 and
    the number of bits for each field, determine the product of f1 and f2.
    Note that we only handle Qm.n, not UQm.n

Input:
-   f1: real number used as operand 1.
-   f2: real number used as operand 2.
-   m: number of bits in the whole number section.
-   n: number of bits in the fraction section.

Return:
-   p: the product of f1 * f2 using Qm.n multiplication.
*/

__attribute__((weak))
float mp_q_mn_multiplication(float f1, float f2, int m, int n) {
    float prod = 0.0;
    return prod;
}
```

Note that we have **planned to use** `uint32_t` to save the encoded data, denoted `D` above. This way, we can easily handle the cases where `uint16_t` or `uint8_t` types are needed.

Note that **you cannot use `prod = f1* f1` directly in the multiplication function, as this is the floating-point multiplication**; what we want to implement is **the multiplication using the approach we discussed in Lctr 13 regarding the Qm.n multiplication**. Note that to make the problem easier, we will confine us to the case of no-greater-than 16-bit multiplications here, which means `m + n <= 15`.

3.3.2 Test cases

To be consistent, we provide the test cases here. The test cases of the first two test functions are duplicated from examples of Lctr 13, as indicated in the code below.

```
void test_mp_uq_and_q_mn_decoding(void) {
    // Case 1: Example 3 of Lctr 13
    // Case 2: Example 5 of Lctr 13
    uint32_t D = 0xAD;
    float exp_f[2] = {21.625, -10.375};
    float act_f[2];

    act_f[0] = mp_uq_and_q_mn_decoding(D, 0, 5, 3);
    act_f[1] = mp_uq_and_q_mn_decoding(D, 1, 4, 3);

    TEST_ASSERT_EQUAL_FLOAT_ARRAY(exp_f, act_f, 2);
}

void test_mp_uq_and_q_mn_encoding(void) {
    float f1 = 3.141593;  // Case 1: Example 4 of Lctr 13
```

```
    float f2 = -3.1234;   // Case 2: Example 7 of Lctr 13
    uint32_t exp_D[2] = {0x3244, 0xFFFFF382};
    uint32_t act_D[2];

    act_D[0] = mp_uq_and_q_mn_encoding(f1, 0, 4, 12);
    act_D[1] = mp_uq_and_q_mn_encoding(f2, 1, 5, 10);

    TEST_ASSERT_EQUAL_UINT32_ARRAY(exp_D, act_D, 2);
}

void test_mp_q_mn_multiplication(void) {
    float f1 = 0.5, f2 = 0.25;
    float exp_p = 0.125;    // Example 8 of Lctr 13
    float act_p = mp_q_mn_multiplication(f1, f2, 0, 15);
    TEST_ASSERT_EQUAL_FLOAT(exp_p, act_p);
}
```

3.3.3 The main App (20 pts)

Once we pass all the test cases, you need to write code in the main App to solve the following problem:

Assume we use Q15 to represent real numbers $f_1 = 0.5$, $f_2 = 0.25$, $f_3 = -0.625$. Perform the following operation in Q15: $f_A = f_1 \times f_2 \times f_3$. Note you need to obtain the following results by calling the above functions:

1. Encode the real numbers in Q15 format to obtain I_1 to I_3.
2. Find the value of f_A using the `mp_q_mn_multiplication` function.
3. Find the value of $f_{A_float} = f_1 \times f_2 \times f_3$ using floating-point multiplication directly.
4. Print out the values of I_1 to I_3 in 16-bit hexadecimal (with `printf` format `0x%04X`).
5. Print out the values of f_A and f_{A_float}. (with `printf` format `%8.7f`).

Note that you need to do the following for this App:

- We need to enable the **printing of floating-point numbers**, as discussed in Section 9.11.2 **Using FPU and floating-point functions** of the Companion.
- You may need to include the necessary header functions.

3.4 Submission of the work (10 pts)

Your report should include the following:

- The code you wrote for the functions to pass the tests.
- The screenshot that shows that you have passed all the tests.
- The screenshot that shows the running result of the App.

Here are what you need to submit:

- Your brief report named as `ee4u-proj3-report-lastname-firstname.pdf`.
- Your project zipped as `ee4u-proj3-lastname-firstname.zip`. Before zipping your project, clean out the builds you have created in CubeIDE.